

L06. Infinite Horizon Dynamic Programming

*Lecturer: Munther Dahleh**Scribe: Jinhee Won*

1 Introduction

In the last few lectures, we covered finite-horizon dynamic programming, finite-horizon LQR, and infinite-horizon LQR. This lecture, we come back to dynamic programming, but instead in the infinite horizon setting.

2 Dynamic Programming Review

Recall that a dynamic programming problem is described by a state, an action, a transition model, and a cost:

$$\begin{aligned}
 s_t &\in \mathcal{S} && \text{(state),} \\
 a_t &\in A_t(s_t) && \text{(action),} \\
 s_{t+1} &= f_t(s_t, a_t, w_t), \quad w_t \sim P_t(\cdot \mid s_t, a_t) && \text{(transition),} \\
 c_t &(s_t, a_t, w_t) && \text{(stage cost).}
 \end{aligned}$$

A feedback policy $\pi = (\mu_0, \dots, \mu_{N-1})$ specifies the action through $a_t = \mu_t(s_t)$. For a finite horizon N , the optimal value function satisfies

$$V_t^*(s) = \min_{a \in A_t(s)} \left\{ \bar{c}_t(s, a) + \gamma \sum_{s'} p_t(s' \mid s, a) V_{t+1}^*(s') \right\}, \quad V_N^*(s) = c_N(s),$$

where $\bar{c}_t(s, a)$ is the expected stage cost and $\gamma \in [0, 1)$ is the discount factor.

In LQR, we followed the same principles and found that the optimal control is linear in the state. We then extended to the infinite horizon by showing convergence as the horizon grows, under suitable assumptions.

We wish to do the same here, but general dynamic programming lacks the linear dynamics and quadratic costs of LQR. Thus, our argument using reachability and observability to prove convergence and stability does not directly apply.

This leads to the next question: what assumptions ensure that a sufficiently long finite-horizon problem approximates the infinite-horizon optimal value?

2.1 First, backward recursion

Before we worry about convergence, we must first address the issue of having no terminal time from which to begin backward recursion.

We introduce the assumption of *stationarity*: the dynamics, stage cost, action constraints, and disturbance distribution do not depend on time. Thus, a problem with N steps remaining is the same regardless of when it starts. We can remove the time subscripts from the model and thus index the value function by the number of steps remaining.

With zero terminal cost, we obtain

$$V_0^*(s) = 0, \quad V_{N+1}^*(s) = \min_{a \in A(s)} \mathbb{E}[c(s, a, w) + \gamma V_N^*(f(s, a, w)) \mid s, a].$$

Instead of working backward from a fixed terminal time, we can now build forward from a zero-step problem to increasingly long horizons.

2.2 Formalizing the question

Now that we can compute values for increasingly long horizons, we ask whether they converge to the infinite-horizon value.

For a policy π , let $V_N^\pi(s)$ denote its expected discounted cost over N steps, starting from state s , with zero terminal cost. Its infinite-horizon value, when the limit exists, is

$$V^\pi(s) = \lim_{N \rightarrow \infty} V_N^\pi(s).$$

The question is twofold: does the optimal finite-horizon value converge to a finite limit, and can we exchange optimization and the limit?

$$V^*(s) = \inf_{\pi} \lim_{N \rightarrow \infty} V_N^\pi(s) \stackrel{?}{=} \lim_{N \rightarrow \infty} \inf_{\pi} V_N^\pi(s) = \lim_{N \rightarrow \infty} V_N^*(s).$$

If this equality holds, a sufficiently long finite-horizon problem approximates the infinite-horizon optimal value.

In LQR, reachability and observability gave sufficient conditions for convergence and stability. Here, we ask what roles they played in the proof and what assumptions can serve those roles for general dynamic programming.

2.3 Proving boundedness (“reachability”)

In LQR, reachability gave an upper bound on the optimal cost. Here, we instead ensure boundedness with two assumptions:

1. Discounting: $0 \leq \gamma < 1$.
2. Bounded stage costs: $|c(s, a)| \leq \bar{C} < \infty$ for all admissible s, a .

Writing $C_t = c(s_t, a_t)$, for a fixed stationary policy μ ,

$$V_N^\mu(s) = \mathbb{E}^\mu[C_0 + \gamma C_1 + \cdots + \gamma^{N-1} C_{N-1} \mid s_0 = s].$$

Thus,

$$|V_N^\mu(s)| \leq \bar{C} \sum_{t=0}^{N-1} \gamma^t \leq \frac{\bar{C}}{1-\gamma} < \infty.$$

The same bound holds for any policy, so the optimal finite-horizon values are also uniformly bounded.

2.4 Proving monotonicity

Assuming nonnegative stage costs and zero terminal cost, the same argument as in LQR applies: adding one more step cannot decrease the total cost. For any policy π ,

$$V_{N+1}^\pi(s) = V_N^\pi(s) + \gamma^N \mathbb{E}^\pi[C_N \mid s_0 = s] \geq V_N^\pi(s).$$

Taking the infimum over policies gives

$$V_{N+1}^\pi(s) \geq V_N^\pi(s).$$

Thus, the values are nondecreasing and bounded above, so they converge to a finite limit,

$$V^\pi(s) = \lim_{N \rightarrow \infty} V_N^\pi(s).$$

2.5 Switching min and lim

Finally, we again run into the question of switching the min and the lim to go from a fixed policy π to the optimal value. In LQR, observability ensured that unstable modes could not remain hidden from the state cost. Here, we instead control the remaining cost uniformly over policies.

Using the same assumptions of discounting and bounded stage costs, for any policy π ,

$$|V^\pi(s) - V_N^\pi(s)| \leq \bar{C} \sum_{t=N}^{\infty} \gamma^t = \frac{\bar{C}\gamma^N}{1-\gamma} \rightarrow 0.$$

Thus, the remaining cost vanishes uniformly over states and policies. Since this bound holds for every policy,

$$V_N^*(s) - \varepsilon_N \leq V^\pi(s) \leq V_N^*(s) + \varepsilon_N.$$

Therefore,

$$V^*(s) = \inf_{\pi} \lim_{N \rightarrow \infty} V_N^\pi(s) = \lim_{N \rightarrow \infty} \inf_{\pi} V_N^\pi(s) = \lim_{N \rightarrow \infty} V_N^*(s).$$

Thus, we can use the recursion in the infinite horizon case

$$V^*(s) = \min_{a \in A(s)} \left\{ c(s, a) + \gamma \sum_{s'} p(s' \mid s, a) V^*(s') \right\}$$

where we assume a stationary model with finite state and action spaces and

$$0 \leq \gamma < 1, \quad |c(s, a)| \leq \bar{C} < \infty.$$

3 Conclusion

Now we can solve the dynamic programming recursion through value iteration! Next class, we will cover a different way of viewing the dynamic programming problem through something called policy iteration, using a tool called the contraction operator.